

Plan — Réduction des coûts d'abord (Chantier C), puis Chantier I (Qwen local), Chantier V (RAG de valorisation) et Chantier M (mentor RAG lutherie)

Date : 2026-09-22 (v3 — réordonné sur l'objectif affiché : **réduire les coûts**. Projet de plaisir, pas de gagne-pain ; contrainte matérielle acceptée : Dell T5810 / RTX 2060 Super 8 Go, pas d'achat de GPU.)

Statut : brouillon à valider

Sources vérifiées : JOURNAL.md (dev, 19 → 22/09),
REPRISE_SESSION_2026-09-21.md ,
COST_OPTIMIZATION_CHANTIERS.md (Chantiers H/I),
compare_qwen_flashlite_agreement.py ,
compare_qwen_local_vs_prod.py ,
install_ollama_qwen_dell.sh , prompts.json ,
geminiChatService.js , RESTORATION_PLAN_PLAN.md ,
CHANTIER_B_PERCEPTION_RAISONNEMENT_PLAN.md .

Complément 22/09 (soir) : le candidat `hybrid` (Qwen décrit sans interpréter → Gemini raisonne sans image, candidates.py:218-241 , 06/09) est le protocole «

texte seul » du projet ; **jamais exécuté**, aucun résultat. Il remplace le candidat `text_only` de la v2 (§1.4). Cache explicite Gemini : écarté, ROI négatif confirmé (`GEMINI_PROMPT_CACHING_PLAN.md` §7.6, ratio 0,13 ; recalcul 22/09 : stockage 0,32–1,46 \$/j contre ~0,07 \$/j d'économie, T1 n'étant plus Gemini).

0. État de départ (22/09/2026)

Sujet	État
Portier T1	Qwen cloud (<code>qwen/qwen3.8-flash</code> , TokenRouter) depuis le 20/09, <code>T1_GATEKEEPER_PROVIDER</code> avec repli <code>gemini</code> sans redéploiement. Flash-Lite en miroir sous <code>flashliteGatekeeper*</code> .
Base de décision	164 annonces comparées (run #53) : 12,8 % « Gemini accepte / Qwen aurait rejeté » ; après lecture des verdicts T2 réels : 1 perte confirmée / 166 . <code>audit_rejected_gems.py</code> : 0 pépite ratée / 165.
Incident	Panne TokenRouter 22/09 23h50 + <code>TOKENROUTER_API_KEY</code> possiblement absente du <code>DOT_ENV</code> . Alerte

Sujet	État
	<code>notify_gatekeeper_failure</code> en place depuis le 20/09.
Chantier I	Ollama + Qwen3-VL-8B Q4_K_M sur le Dell T5810 (100.94.33.54:11434), VRAM 6982/8192 Mio après chargement. <code>compare_qwen_local_vs_prod.py</code> écrit, jamais exécuté (doit tourner sur le ThinkCentre, Postgres localhost).
Chantier A (Postgres/API)	Branche isolée <code>claude/firestore-postgres-migration</code> , staging seulement, jamais mergée ; dev rapatrié <code>backend/api/*</code> (étapes 1–2/5, 21/09). Pas en prod.
Détecteur maison	Pipeline YOLO-OBb phase 1 terminée (Label Studio), OWLv2 phase 0 validée sur le Dell (19/08). Fournisseur de crops, non modifié ici.
Objection ouverte (utilisateur)	Ni Flash-Lite ni Qwen n'ont de taux d'erreur mesuré contre une vérité indépendante de Gemini.
Non-déterminisme	Même annonce, même prompt : <code>deal_score 9</code> puis <code>< seuil</code> (Acoustasonic, runs #48/#49).

Proposition externe « VLM spécialisé lutherie » — ce qui en reste

Retenu : modèle Qwen-VL léger en local ; LoRA par distillation ; base de connaissances lutherie consultée à l'inférence ; crops HD des zones critiques ; banc d'évaluation chiffré.

Rejeté : Qwen 7B/8B entraîné sur la carte 8 Go (impossible avec images, voir §2.5) ; Grounding DINO (détecteur maison existant) ; « le VLM redimensionne en 448×448 » (faux pour Qwen2.5/3-VL, résolution dynamique) ; dataset synthétique (la prod fournit un corpus réel, rejets T1 avec reasoning inclus) ; V-RAG d'authenticité (biais des voisins, photos ~650×960 sans les détails invoqués) ; RAG brevets/organologie **dans la cascade** (déplacé vers le mentor) ; le local identifie/valorise (incident Guerrilla Guitars) ; cibles > 98 % sans baseline.

Principe directeur

L'objectif est la facture. Chaque chantier est classé par ce qu'il rapporte en dollars, et ce qui n'en rapporte pas est dit tel quel.

Chantier	Effet sur la facture	Nature
C — Coûts (dashboard,	Le seul avec un gain à deux chiffres par mois	Économie

Chantier	Effet sur la facture	Nature
chat, miroir, T3)		
I – Qwen local	1–3 \$/mois (tokens d'images) + secours en cas de panne fournisseur	Résilience, plaisir
V – RAG de valorisation	Neutre à légèrement positif (autorise un T2 moins cher)	Qualité de détection
M – Mentor RAG	Coût en plus (600–900 tokens d'extraits par message)	Plaisir, apprentissage

Ordre : C d'abord, parce qu'il est le moins cher à faire et le seul qui compte pour l'objectif. I, V, M ensuite, en sachant pourquoi on les fait. **Le T2 n'est jamais remplacé par un modèle maison pour des raisons de coût** : il coûte quelques dollars par mois ; un T2 local ne se justifierait que par l'indépendance ou la curiosité (§2.9).

I est indépendant des autres. V et M partagent l'infrastructure (index, embeddings) : construite une fois

pour V, réutilisée par M.

1. Chantier C – Réduction des coûts

1.1 Point de départ

Facture août 2026 : ~102 \$, dont ~16 \$ Firestore et ~85 \$ Gemini. Le T1 ($\approx 34\%$ de la part Gemini) est parti chez Qwen le 20/09 ($\sim 66,5\%$ /appel). Le T2 coûte quelques dollars par mois (≈ 11 appels/jour). Le T3 tourne sur $\sim 23\%$ des T2. **La ventilation exacte du reste (chat, miroir, T3) n'existe pas** : c'est le premier trou à boucher.

1.2 C-0 – Tableau de bord de coût par poste (un après-midi, zéro appel IA)

Postes : T1 Qwen, miroir Flash-Lite, T2, T3, chat (par modèle), Firestore, Storage. Sources déjà en place : logs `[tokens]` d' `analyzer.py` (modèle, images, in/out/cached), `usage_metadata` logué côté chat (`TOKEN_OPTIMIZATION_PLAN.md`), export de facturation Google Cloud, `analyze_funnel_by_user.py` . Sortie : un script lecture seule + une carte dans le tableau de bord admin, coût/jour par poste, relancé chaque semaine. **Aucune optimisation ne démarre avant ce chiffre.**

1.3 C-1 – Le chat (probablement la première ligne)

Le chat par annonce tourne sur le modèle le plus cher, avec photos, et rejoue l'historique à chaque message (bug de rejeu intégral des photos corrigé le 23/08, mais l'historique texte est toujours rejoué).

1. Confirmer 1024 px sur les photos du chat et le nombre de photos envoyées par tour.
2. **Résumé glissant de l'historique** au-delà de N tours (ex. 8) : le client envoie un résumé + les derniers tours au lieu de tout ; le résumé est produit par un modèle bon marché et persisté à côté de l'historique.
3. **Modèle par défaut moins cher, Pro à la demande** : un bouton « Approfondir » (ou déclenchement automatique quand la question touche réparation/valeur) bascule sur Pro pour ce tour seulement. Les questions de routine (« c'est quoi ce chevalet ? ») ne justifient pas 2 \$/M en entrée.
4. Mesurer avant/après avec C-0.

1.4 C-2 — Le miroir Flash-Lite

Tant que l'observation miroir est active, l'ancien T1 est payé sur chaque annonce. Décision : fixer une date de fin d'observation (ex. 300 annonces comparées ou 30 jours), puis `T1_OBSERVATION_ENABLED=false` comme régime normal, réactivable pour une campagne de mesure. Vérifier que la désactivation du 22/09 (faite à la main pendant la panne TokenRouter) est bien l'état voulu et documenté dans le `.env` de prod.

1.5 C-3 – Le T3

Pro-preview à 2/12 \$. Vérifier le seuil de déclenchement (`deal_score` ≥ 8) contre les verdicts T3 réels des 3 derniers mois : combien de T3 ont changé la décision finale ? Et tester la sortie compacte (`gemini_pro_compact` existe déjà dans le harnais) : la moitié de la facture T3 est en sortie.

1.6 C-4 – Firestore

Chantier A, déjà engagé (branche isolée, `backend/api/*` en cours de rapatriement sur `dev`). 16–20 \$/mois, tendance haussière. Hors périmètre de ce plan, mais c'est la deuxième ligne.

1.7 C-5 – Le T2

Choix du fournisseur par le rejeu V-2 (§3.5, phase V-2). Ligne à ~5 \$/mois : ne pas y passer plus d'un après-midi de test.

1.8 Ce qui ne rapporte rien et qu'on fait quand même

Le local (I) et le mentor (M) ne réduisent pas la facture. Ils sont dans ce plan parce que le projet est un plaisir et une école, pas parce qu'ils sont rentables. Le dire évite de leur consacrer trois semaines en croyant économiser 5 \$.

2. Chantier I étendu — Qwen local

2.1 Rôles possibles du modèle local, par ordre d'engagement

1. **Secours** (aucun fine-tuning) : chaîne T1 `qwen_cloud` → `qwen_local` → `gemini`. Couvre une panne TokenRouter sans remonter la facture Gemini. Un local à 70–80 % d'accord vaut mieux qu'un fail-open à 100 % de promotion T2.
2. **Perception** (Chantier B §8) : le local décrit les photos (contrat de perception, §1.6), les tiers de raisonnement ne voient plus d'images. Retire la vision de ~95 % des appels payants.
3. **Portier spécialisé** (LoRA distillé) : le local décide T1 seul. Uniquement si (1) et (2) ont mesuré ce dont il est capable.

Aller directement à (3) — ce que proposait le document externe — saute deux mesures qui coûtent zéro appel payant.

2.2 Phase I-0 — Rejeu réel, corrigé avant lancement (cette semaine)

Le script `compare_qwen_local_vs_prod.py` est le bon protocole (même angle que `compare_qwen_flashlite_agreement.py`, « Cloud accepte / Local aurait rejeté » comme seul cas qui compte). Trois corrections **avant** le premier run, sinon

on mesure un modèle étouffé plutôt qu'un modèle trop petit :

- **VRAM** : 1,2 Go de marge sur le 8B, avec un prompt Portier qui envoie jusqu'à 8 photos ~650×960 → dépassement quasi certain. Ollama ne plante pas : il décharge sur CPU en silence (22 s → minutes) ou tronque le contexte. Fixer `num_ctx` explicitement (Modelfile), plafonner à **4 images / 1024 px** dans ce script, logger `ollama ps` (colonne PROCESSOR, 100% GPU attendu) pendant le run.
- **Second candidat** `qwen_local_4b` (`ollama pull qwen3-v1:4b`, ≈3 Go de marge). Si le 4B non étouffé égale le 8B étouffé, c'est le 4B qui devient la base de tout le reste.
- **Métriques à ajouter au résumé JSON** : taux de JSON valide au schéma strict (`T1_GATEKEEPER_OPENAI_JSON_SCHEMA`), latence médiane et P90 par annonce, taux de statuts hors enum. Sans ça, un écart d'accord peut être instrumental (rejet du format) et non sémantique — leçon du run #42.

Cible : `--limit 15` d'abord (validation technique), puis **≥ 150 annonces** (même ordre de grandeur que le run #53) pour un chiffre comparable.

Lecture attendue (l'utilisateur anticipe de mauvaises performances) : le chiffre à regarder n'est pas l'accord global, c'est la part des « Cloud accepte / Local rejette »

dont le verdict T2 réel est `PEPITE` / `FAST_FLIP` scorés — même filtre qu'au 20/09. Si $\leq 2\%$ des annonces comparées, le rôle **secours** est acquis même avec un accord global médiocre.

2.3 Phase I-1 — Secours (1 semaine, si I-0 $\leq 2\%$ de pertes confirmées)

- `config.py` : `T1_GATEKEEPER_PROVIDER` accepte une **liste ordonnée** (`qwen`, `qwen_local`, `gemini`) ; `_call_t1_provider` (déjà point d'appel unique depuis le 20/09) enchaîne sur échec réseau/timeout/JSON invalide.
- Timeout local court (30 s) : le Dell est partagé avec MoneyBot (Ray), pas dédié, et `run_script_dell.yml` n'a pas été conçu pour un service 24/7. Service Ollama en systemd sur le Dell avec `OLLAMA_KEEP_ALIVE` long pour éviter le rechargement du modèle (plusieurs secondes) à chaque annonce.
- Le miroir `_run_t1_shadow_observation` suit déjà « le fournisseur qui n'est pas le décideur » : quand le local décide (panne cloud), le miroir tombe sur Gemini — comportement correct sans changement.
- `analyze_funnel_by_user.py` : ajouter une ligne de coût `t1_local = 0` explicite, pour que le tableau de bord montre la part des décisions rendues en local.

2.4 Phase I-2 — Perception locale et question « texte seul » (3 semaines)

Prérequis : le contrat de perception (§1.6) écrit et figé.

Mesure « texte seul » d'abord, avec le candidat `hybrid` existant, mais en rejeu réel et non dans le harnais (5 questions jugées par Claude, déjà jugé trop petit et biaisé — Chantiers F/I). Protocole, calqué sur `compare_qwen_local_vs_prod.py` :

1. ~150 annonces avec verdict T2 déjà en base.
2. Qwen (cloud, puis local) décrit les photos avec le prompt hybride existant (état physique + OCR/logo sans interprétation) — prompt conservé tel quel.
3. Rejeu du **T2** (Flash, celui qui voit les photos en prod — pas Pro) avec la description seule, `images=[]`.
4. Comparaison au verdict d'origine ; chiffre qui compte : annonces jugées `PEPITE` / `FAST_FLIP` scorées avec photos et ratées sans. 3 tirages pour tout score proche du seuil T3.

Coût : ~150 appels Qwen + ~150 appels Flash, quelques dollars. Risque documenté :

`prompts.json:331` fait de l'examen du logo la preuve prioritaire — surveiller les rejets

`REJECTED_ITEM` (accessoires, étuis seuls) et les pépites de luthier.

Coût, recalculé 22/09 : bloc statique T2 $\approx$ 4 500 tokens (taxonomie $\sim$ 900, prompt $\sim$ 2 000, few-shots $\sim$ 1 500) ; images $\approx$ 520 tokens/photo à 650×960

(tuiles 768², 258 chacune ; forfait

`media_resolution` Gemini 3 possible, non vérifié sur vos appels). Le hybride retire les images (1 500–4 000 tokens) et les remplace par ~300–500 tokens : **0,2–0,6 cent/annonce T2** de gain, moins l'appel Qwen. Réel mais modeste : le hybride se justifie par la qualité (lire ≠ interpréter) et par des yeux locaux gratuits, pas par la facture. Le seul levier sur le bloc statique est de le **raccourcir** (audit des few-shots) ou de payer moins le token (T2 chez Qwen cloud) – pas le cache explicite (ROI négatif, §0).

Si le T2 aveugle égale le T2 avec photos à ≤ 2 % de pertes, la perception locale peut se limiter à **enrichir le texte** plutôt qu'à décrire exhaustivement – contrat plus court, latence plus faible.

Ensuite :

1. Le local produit le contrat de perception sur les photos (4 max, crops du détecteur YOLO-OBb quand disponibles pour tête/plaque/étiquette).
2. **Shadow** une semaine : T1/T2 continuent de voir les photos ; on stocke la perception locale à côté et on rejoue T2 en texte seul sur un échantillon (coût ~0,008 \$/réanalyse, mesuré) pour comparer verdict et `deal_score` .
3. Bascule T1 puis T2 en « perception locale + raisonneur texte », avec repli photos si la perception est invalide ou absente.

4. Couvrir `force_expert=True` et `analyze_deal_light()` qui sautent T1 (conséquence 8.1 du plan B).

2.5 Phase I-3 — Spécialisation par distillation (optionnelle, 3–4 semaines)

Seulement si I-2 laisse un écart mesuré que le zero-shot ne comble pas.

- **Le fine-tuning du 8B sur le Dell est exclu** : 7 Go en inférence Q4 ; QLoRA avec images $\geq 10-12$ Go. Le « ~6,5 Go Unsloth » cité le 20/09 vaut pour du texte court. Options : (a) entraîner le **4B ou 2B** sur le Dell (fp16, Turing, lent, batch 1, `max_pixels` bas), (b) **entraîner dans le cloud** (2–8 \$/run, chiffré le 20/09) et **héberger en local** — ce qui règle l'objection des ~8 \$/GPU-h d'hébergement. Recommandation : (b).
- **Teacher $\neq$ T1**. Le Portier actuel est Qwen cloud, dont la fiabilité absolue n'est pas établie. Labels d'entraînement : le **verdict T2/T3 final** sur les annonces promues, plus les rejets T1 **confirmés** par `audit_rejected_gems.py`. On apprend « ce que la cascade a fini par décider ». Export dédié qui **inclut** les rejetés (contrairement à `export_dataset_a.py`), avec le `reasoning` T1 stocké depuis `handle_deal_found()`.
- Garde-fou anti-interprétation généralisé dans les cibles (TODO ouvert du 07/09), pas seulement le logo.

- LoRA r = 16–32, ViT gelé au premier run, export GGUF Q4_K_M, retour sur le Dell, rejeu I-0 complet.

2.6 Contrat de perception (à écrire avant I-2 ; fusionne le TODO « tagging des photos par le Tier 1 » du 06/09)

Squelette à compléter : par image `vue` (enum : `corps_face`, `corps_dos`, `tete_face`, `tete_dos`, `manche`, `chevalet`, `rosace`, `etiquette`, `plaque_serie`, `ampli`, `etui`, `autre`), `qualite`, `texte_lu` (transcription exacte + confiance, jamais une marque), `formes` (enum `corps/tête`), `materiel_visible`, `finition`, `etat` (observation + gravité visuelle + zone), `non_visible`.

Synthèse 3–5 phrases sans marque, prix, année, authenticité. Enums fermés, schéma strict, mêmes règles que `T1_GATEKEEPER_OPENAI_JSON_SCHEMA`.

2.7 Jeu d'évaluation permanent et vérité indépendante

Réponse partielle à l'objection ouverte : les seules étiquettes qui ne viennent pas d'un modèle Gemini sont **les actions de l'utilisateur** — favoris, `isPurchased`, plan de restauration ouvert, annonces `NOT_PROMOTED` repromues à la main — et les **annonces vendues**, avec une réserve : la date de vente est déduite d'une disparition d'annonce, pas d'une transaction, donc bruitée ; ne garder que celles dont la date est plausible.

Petit et biaisé vers les acceptés, mais c'est le seul rappel mesurable sur de vraies pépites.

- Export lecture seule →
backend/benchmark/eval_set.json (remplace le dataset.json à 5 cas), gelé, exclu de tout dataset d'entraînement.
- Tout audit dont un score est proche du seuil T3 : **3 tirages** (non-déterminisme mesuré aux runs #48/#49), décision à la médiane.

2.8 Risques

Risque	Mitigation
Dell partagé (MoneyBot/Ray), pas un serveur	Timeout court, repli Gemini, OLLAMA_KEEP_ALIVE , systemd ; ne jamais faire du local le seul décideur sans repli.
Local étouffé en VRAM pris pour « trop petit »	num_ctx fixé, 4 images, ollama ps loggé, 4B en second candidat.
Distiller les erreurs du T1	Teacher = verdict final de cascade + rejets confirmés, jamais le T1 seul.
Perception « texte seul » perd les pépites visuelles	Shadow + rejeu T2 avant bascule ; repli photos sur perception invalide.

Risque	Mitigation
<code>TOKENROUTER_API_KEY</code> disparaît à nouveau du <code>DOT_ENV</code>	Vérification au démarrage du bot + alerte dédiée (déjà en partie via <code>notify_gatekeeper_failure</code>).

2.9 I-4 – T2 local (curiosité, pas un levier de coût)

Le T2 coûte quelques dollars par mois : un T2 local n'économise rien. Il devient **techniquement** envisageable seulement après I-2 (perception) et V (comparables), quand le T2 est un raisonneur texte. Sur 8 Go de VRAM sans achat de GPU : un MoE à faible activation (Qwen3.6-35B-A3B, Q4 $\approx$ 20 Go, poids en RAM, couches partagées sur GPU, quelques tokens/s) ou un dense 27B partagé CPU/GPU (1–3 tokens/s). 2 à 10 minutes par verdict – acceptable à $\sim$ 11 appels/jour, à condition d'une file séquentielle (incompatible en mémoire avec le 8B de perception) et d'aucune collision avec MoneyBot. Candidat `t2_local` dans le jeu V-2, même métrique que les autres. À faire pour l'indépendance et le plaisir, jamais présenté comme une économie.

3. Chantier V – RAG de valorisation

3.1 Pourquoi

Le T2 produit `estimated_value` / `resale_potential` depuis la mémoire du modèle. Conséquences : valeurs non auditables, périssables, non reproductibles (non-déterminisme mesuré, Acoustasonic runs #48/#49), et un risque spécifique si le T2 passe chez Qwen (culture des prix du marché de l'occasion nord-américain probablement plus faible que Gemini). Une erreur de valorisation se propage directement au `deal_score`, donc aux seuils T3, donc à la détection de pépites.

Écarté (v1 de ce plan) : une table de prix agrégée depuis les annonces vendues. Deux défauts : la date de vente est déduite d'une disparition d'annonce (bruitée) ; le jeu est concentré sous ~400 \$ (plafond historique des recherches), donc vide là où l'erreur coûte le plus (haut de gamme, déclenchement T3).

3.2 Principe : le T2 lit le marché au lieu de le deviner

Avant l'appel T2, une étape de récupération sort **N comparables** (8 par défaut) et les injecte dans le prompt :

```
« Comparables récents (source, date, prix, état, titre) :  
[...]. Estime la valeur en t'ancrant sur ces  
comparables. S'ils sont absents, trop anciens ou non  
comparables, dis-le et baisse confidence. »
```

Le JSON de sortie ne change pas (mêmes champs) ; seule la source de la valeur change. Ajouter un champ `valuation_basis` (`comparables` / `comparables_faibles` / `memoire_modele`) pour tracer d'où vient chaque estimation.

3.3 Architecture hybride (pas un RAG de texte)

La valorisation est un problème **structuré** ; la recherche sémantique seule renverrait des annonces « qui parlent pareil » sans respecter prix, état ni date.

- **Filtre SQL (exact)** : catégorie taxonomique (même famille), fenêtre de dates (18 mois par défaut, élargie si trop peu de résultats), monnaie normalisée en CAD, exclusion des rejets `REJECTED_ITEM` / `REJECTED_SERVICE` , exclusion de l'annonce elle-même et de ses doublons (même vendeur/titre).
- **Rapprochement flou (embeddings)** : sur `marque + modèle normalisé + titre` , pour rapprocher « yamaha f g 180 rouge » de « FG-180 vintage ». bge-m3 sur CPU (multilingue, titres FR/EN mêlés).
- **Re-classement** : score combiné similarité × récence × proximité d'état, puis diversité des sources (pas 8 fois le même revendeur).
- **Sortie** : comparables + statistiques (médiane, P25–P75, n, âge moyen) + un indicateur de couverture.

Stockage : **SQLite + sqlite-vec** tant que Postgres n'est pas en prod (Chantier A), migration **pgvector** ensuite. Même index physique que le mentor (Chantier M), collections distinctes.

3.4 Sources de comparables, de la plus solide à la plus fragile

1. **Scans propres (toutes annonces, pas seulement les vendues)** : ~6 500 annonces avec prix demandé, titre, date de scan, marque/modèle/classification T2, photos. Prix demandé $\neq$ prix vendu, mais pour un `deal_score` **relatif** (« 40 % sous les comparables du mois ») c'est la bonne référence ; dense dans la gamme de chasse ; gratuit ; mis à jour à chaque scan. Signal additionnel faible : durée de vie de l'annonce (une annonce qui disparaît vite était probablement bien placée) — à utiliser comme poids, jamais comme date de vente.
2. **Comparables externes pour le haut de gamme** : Reverb (API publique, guide de prix — **lire les conditions d'utilisation avant ingestion**), occasions des détaillants canadiens, ventes terminées eBay. Ingestion **périodique par lot** (hebdomadaire), jamais à la volée pendant une analyse. Priorité aux familles/marques présentes dans la taxonomie et aux fourchettes > 400 \$ où les scans propres sont vides.

3. **Fiches de référence par modèle** (années de production, usines, fourchettes connues, versions recherchées) : partagées avec le mentor (§4.3) ; servent de garde-fou quand les comparables sont rares (« cette série vaut rarement moins de X »).

3.5 Phases

- **V-0 – Couverture (1 semaine, lecture seule, zéro appel IA)** : sur les 6 500 annonces, pour chaque annonce promue en T2, compter combien de comparables le filtre SQL + un rapprochement naïf trouveraient. Résultat attendu : carte de couverture par famille × tranche de prix. Décide où les sources externes sont indispensables.
- **V-1 – Index et récupération (2 semaines)** : normalisation marque/modèle (réutiliser la canonicalisation taxonomique existante `backend/taxonomy.py`), embeddings, requête hybride, script CLI « donne-moi les comparables de l'annonce X ». Relecture manuelle de 30 cas (dont ta gamme de chasse et 10 cas > 400 \$).
- **V-2 – Rejeu T2 avec comparables (1–2 semaines)** : ~150 annonces déjà analysées, T2 rejoué avec et sans comparables, **même modèle**, 3 tirages sur les cas proches du seuil. Candidats T2 évalués dans le même rejeu : Gemini 3.7 Flash (référence), Qwen3.7 Plus, Qwen3.8 Flash — c'est ici que se tranche le

remplacement de Gemini 3.7 Flash avant son passage à 1,50/7,50 \$ au 01/01/2027.

- **V-3 – Ingestion externe (2–3 semaines, selon V-0) :** connecteurs par lot pour les sources retenues, dédoublonnage, normalisation CAD.
- **V-4 – Bascule :** comparables injectés en prod au T2 puis au T3, avec `valuation_basis` loggé ; repli sur le comportement actuel si la récupération échoue.

3.6 Métriques

- **Stabilité :** écart-type de `estimated_value` sur 3 tirages, avant/après (attendu : forte baisse – c'est le gain le plus sûr).
- **Justesse :** erreur relative médiane sur l'étalon disponible – achats réels (`isPurchased` + prix payé) et vendus à date plausible ; petit, mais indépendant de Gemini.
- **Couverture :** part des analyses T2 avec `valuation_basis = comparables` (≥ 5 comparables à moins de 18 mois).
- **Honnêteté :** sur les cas sans comparables, part des sorties qui le déclarent et baissent `confidence` (cible ≥ 90 %).
- **Effet aval :** pertes confirmées de pépites (même définition que le Chantier H) $\leq$ niveau actuel.

3.7 Risques

Risque	Mitigation
Prix demandés gonflés (vendeurs optimistes)	Utiliser médiane et P25, pas la moyenne ; pondérer par durée de vie de l'annonce.
Comparables mal appariés (même marque, autre gamme)	Filtre par classification taxonomique fine + relecture V-1 ; le T2 voit les titres et peut écarter.
Haut de gamme sans données	V-0 le mesure ; sources externes ciblées ; sinon <code>valuation_basis = memoire_modele</code> explicite.
Conditions d'utilisation des sources externes	Lecture avant ingestion ; usage interne, jamais de republication.
Tokens en plus au T2 (8 comparables ≈ 600–900 tokens)	Format compact ; compensé si le T2 passe chez Qwen.

4. Chantier M – Mentor lutherie (RAG sur le chat frontière)

4.1 Objectif

Faire du chat par annonce un mentor de réparation et d'histoire de l'instrument, **fiable et sourcé**, pour un apprenti luthier sans cours. Modèle frontière du chat (Gemini 3.1 Pro aujourd'hui, Sonnet 5 si Chantier F) ; l'index est agnostique au modèle.

4.2 Mécanisme (le cloud n'est pas un obstacle)

Le RAG est une recherche faite de notre côté dont le résultat est injecté dans le prompt. Documents découpés (~400 tokens, tableaux préservés comme unités), vectorisés par **bge-m3 sur CPU**, index local ; à chaque question, top-k (≤ 6) avec source, ajoutés au message avec la consigne « appuie-toi dessus, cite, dis quand ça ne couvre pas ».

Intégration au chat 100 % client-side (Firebase AI Logic) :

- **Variante 1 (cible)** : outil
`search_lutherie_docs(query, k)` déclaré à côté de `request_photo_review /`
`propose_restoration_step` ; le client appelle un endpoint backend authentifié (Firebase Auth) ; retour en `functionResponse` ; même discipline de fermeture de tour que l'existant.
- **Variante 2 (prototype)** : pré-récupération à chaque message, injection dans le texte.

Hébergement : `backend/api/*` (FastAPI) est en cours de rapatriement sur `dev` (étapes 1–2/5) mais **pas en**

prod. Deux options : (a) attendre l'étape 5/5 et ajouter une route `/rag/search` ; (b) ne pas dépendre du Chantier A et passer par le bus de commandes Firestore existant (`commands` , réponse écrite dans une sous-collection lue par le client). Recommandation : (b) pour M0/M1, (a) une fois l'API en prod. Index : le même que le Chantier V (SQLite + sqlite-vec, puis pgvector), collection distincte ; s'il est déjà construit pour V, M n'ajoute que le corpus.

4.3 Corpus (indexé avec `source` , `titre` , `section` , `url_ou_fichier` , `type`)

1. **Réparation/réglage** : frets.com (Frank Ford), articles StewMac / Trade Secrets, guides d'entretien constructeurs (Fender, Gibson, Taylor, Martin, Yamaha), livres de lutherie possédés, scannés pour usage personnel.
2. **Specs d'usine par modèle** : diapason, sillet, rayon, électronique, matériaux, années/changements de specs. `pdfplumber` , tableaux préservés.
3. **Histoire/généalogie** : Wikidata/SPARQL, pages constructeurs, références. Matière de la fiche instrument (§4.4, phase M3).
4. **Sécurité de l'instrument** : 10–20 fiches rédigées à la main (truss rod : amplitude et sens ; colles et chaleur ; ponçage de touche ; nitro) marquées `type: securite` , **toujours** injectées quand la question

touche un geste irréversible, indépendamment du score de similarité.

- 5. **Expérience propre** : plans de restauration terminés (étapes, `actualCost`, notes) indexés comme corpus personnel.

4.4 Phases

- **M0 – Prototype CLI (1 semaine)** : indexer frets.com + 2–3 manuels ; 20 questions réelles tirées de tes plans de restauration ; comparer avec/sans RAG (cite une source ? dit « je ne trouve pas » hors corpus ?).
- **M1 – Intégration chat (2 semaines)** : recherche via bus de commandes ou API ; outil `search_lutherie_docs` (variante 1) avec repli sans `tools` ; sources repliables dans la bulle ; corpus sécurité forcé.
- **M2 – Plan de restauration augmenté (1–2 semaines)** : « Demander conseil » construit la requête RAG depuis `label` + `category` + `notes` + observations d'état ; « Faire le point » injecte `done` / `actualCost` + extraits pour le reste ; `propose_restoration_step` doit citer la source qui justifie l'étape et le coût.
- **M3 – Fiche instrument (1–2 semaines)** : générée une fois au passage `isPurchased = true` (ou à la demande), persistée en sous-collection `guitar_deals/{dealId}/instrumentSheet`.

Histoire, specs d'usine de la période, points de vigilance connus, lecture du numéro de série si visible, références. Un appel frontière amorti sur la vie de l'instrument.

4.5 Métriques

Réponses techniques citant ≥ 1 source : ≥ 90 %. Hors corpus : ≥ 80 % de « je ne trouve pas dans les sources » sur 20 questions pièges. Gestes irréversibles : 100 % rappellent la limite (juge sur 20 questions). Tokens/message suivis dans le tableau de bord admin.

4.6 Risques

Fausse assurance (extraits hors sujet cités avec aplomb) → seuil de similarité, consigne explicite, sources visibles. Conseils dangereux → corpus sécurité forcé, jamais d'écriture automatique dans le plan. Droits → usage personnel, index non partagé. Tokens → chunks courts, $k \leq 6$, variante 1.

5. Ordonnancement

```
Sem. 1      C-0 dashboard de coût (lecture  
seule)      I-0 rejeu corrigé (8B + 4B)      V-0  
couverture  
Sem. 2      C-1 chat (1024 px, résumé  
glissant)    I-1 secours  
V-1 index + récupération
```

Sem. 3 C-1 modèle par défaut + Pro à la demande ; C-2 fin du miroir ; C-3 T3
Sem. 4-5 contrat de perception ; I-2 hybrid
V-2 rejeu T2 (+ choix fournisseur, avant 01/01/2027)
Sem. 6-8 I-2 shadow / bascule
V-3/V-4 ; M0-M1 sur le chat déjà allégé
Ensuite I-3 / I-4 si envie
M2-M3
Chaque semaine : relire C-0 et recalculer avant d'attaquer la ligne suivante.

Chantier F (Sonnet 5 au T3) : sans impact sur I (perception agnostique), V et M (index agnostiques au modèle). V bénéficie au T3 autant qu'au T2. Le benchmark T3 doit tester les raisonneurs **avec la même entrée de perception**.

6. Décisions à prendre

0. **C-0 avant tout** : construire la ventilation de la facture par poste. Puis, seulement, choisir la ligne à attaquer.
 - 0b. Le mentor (M) est-il accepté comme coût de plaisir, à construire sur le chat déjà allégé (C-1) ?
1. Lancer I-0 avec les trois corrections (§1.2) ou tel quel ? Recommandation : corrigé, sinon le chiffre ne vaut rien.
2. Le rôle **secours** est-il un objectif en soi ?
Recommandation : oui, c'est le seul qui ne dépend

pas de la qualité du local.

3. Lancer le rejeu réel du candidat `hybrid` (§1.4) avant toute décision sur la perception ; auditer les 1 500 tokens de few-shots du prompt T2.
4. M1 et V par bus de commandes / appel direct depuis le bot (indépendant du Chantier A) ou attendre l'API en prod ? Pour V, la récupération tourne dans le bot, avant l'appel T2 : aucun endpoint nécessaire.
5. Export du jeu d'éval permanent (§1.7) : quelles actions utilisateur comptent comme « pépite confirmée » ?
6. Chantier V : quelles sources externes, après lecture de leurs conditions d'utilisation (Reverb, détaillants, eBay) ?

7. Hors périmètre

Chantier F (T3), Chantier A (migration), détecteur YOLO-
OBB (fournisseur de crops, inchangé), Chantier G
(routage recherche active).